

PROBLEME I

Question 1:

On retrouve la méthode d'Euler pour $(\alpha, \beta) = (1, 0)$

Question 2:

```
def Heun(f,t0,y0,T,N) :
    t,h,y = t0,T/N,y0
    tt,yh = [t0],[y0]
    while t <= t0+T :
        y += (h/2)*(f(t,y)+f(t+h,y+(h*f(t,y))))
        t += h
        tt.append(t)
        yh.append(y)
    return tt,yh
```

Question 3:

```
def Euler(f,t0,y0,T,N) :
    t,h,y = t0,T/N,y0
    tt,yh = [t0],[y0]
    while t <= t0+T :
        y += h*(f(t,y))
        t += h
        tt.append(t)
        yh.append(y)
    return tt,yh
```

Question 4 :

La complexité de la fonction Euler : $O(N)$

Question 5 :

$$\begin{cases} y_0(t) = y_1(t) \\ y_1'(t) = -4\pi^2 \cos(2\pi t) - y_1(t) - y_0(t) \end{cases}$$

Question 6 :

$$F(t,X) = (X[1] , -4\pi^2 \cos(2\pi t) - X[1] - X[0])$$

Question 7 :

PROBLEME II

Question 8 :

```
def indice(c, ch) :
    for i in range(len(ch)) :
        if ch[i] == c : return i
    return -1 # n'existe pas

def enMajuscule(CH) :
    alphabet_min = 'abcdefghijklmnopqrstuvwxyzâàéèëïîôùüÿç'
    alphabet_maj = 'ABCDEFGHIJKLMNOPQRSTUVWXYZAAEEEEIIIOUUUYC'
    s = ''
    for c in CH :
        i = indice(c, alphabet_min)
        if i != -1 : s += alphabet_maj[i]
        else : s += c
    return s
```

Question 9 :

```
def majusculesSeules(CH) :
    alphabet_min = 'abcdefghijklmnopqrstuvwxyzâàéèëïîôùüÿç'
    alphabet_maj = 'ABCDEFGHIJKLMNOPQRSTUVWXYZAAEEEEIIIOUUUYC'
    s = ''
    for c in CH :
        i = indice(c, alphabet_min)
        if i != -1 : s += alphabet_maj[i]
    return s
```

Question 10 :

```
def vigenereEncode(CH, CL) :
    alphabet = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
    n = len(CH)
    m = len(CL)
    ch_cryptée = ''
    for i in range(n) :
        lettre_clé = CL[i%m]
        k = indice(CH[i], alphabet)
        h = indice(lettre_clé, alphabet)
        p = (k+h)%26
        ch_cryptée += alphabet[p]
    return ch_cryptée
```

Question 11 :

La complexité de la fonction **vigenereEncode** est $O(\text{len}(\text{CH}))$

Question 12 :

```
def vigenereEncodeRec(CH, CL, i) :
    alphabet = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
    n = len(CH)
    m = len(CL)
    if i > n : return ''
    lettre_clé = CL[i%m]
```

```

k = indice(CH[i],alphabet)
h = indice(lettre_clé,alphabet)
p = (k+h)%26
return alphabet[p] + vigenereEncodeRec(CH,CL,i+1)

```

Question 13 :

Question 14 :

```

def vigenereDecode(CH,CL) :
    alphabet = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
    n = len(CH)
    m = len(CL)
    ch_originale = ''
    for i in range(n) :
        lettre_clé = CL[i%m]
        k = indice(CH[i],alphabet)
        h = indice(lettre_clé,alphabet)
        p = (k-h)%26
        ch_originale += alphabet[p]
    return ch_originale

```

Question 15 :

Afin que le message crypté soit robuste, la clé choisie doit être assez longue et variée.

Question 16 :

```

def sousChaines(CH,d,p) :
    n = len(CH)
    s = ''
    l = 0
    while d+(l*p)<n : s += CH[d+(l*p)]
    return s

```

Question 17 :

```

def listeSousChaines(CH,p) :
    L = []
    For d in range(p) : L.append(sousChaines(CH,d,p))
    return L

```

Question 18 :

```

def frequencesCaracteres(CH) :
    alphabet = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
    L = []
    for c in alphabet :
        n = 0
        for x in CH :
            if c == x : n += 1
        L.append(n)
    Return L

```

Question 19 :

```
def code(CH,p) :  
    alphabet = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'  
    clé = ''  
    LSC = listeSousChaines(CH,p)  
    for s in LSC :  
        L = frequencesCaracteres(s)  
        i = indiceMax(L)  
        clé += alphabet[(6-i)%26]  
return clé
```

----- FIN DE L'EPREUVE -----